

Nestboard

Room Automation System

Frontend Technical Documentation

A cross-platform mobile client for the ESP32-based room-automation stack, built with React Native and Expo.

Mobile Application & Frontend developed by

Abdullah Amir

2024-CS-156

Project: ESP Room Automation
Module: Frontend (`app/SmartOffice`)
Author: Abdullah Amir (2024-CS-156)
Platform: React Native · Expo · TypeScript
Course: Computer Organization & Assembly Language (COAL)
Session: Semester 4 · 2024 · UET

Contents

1	Introduction	2
1.1	Feature Summary	2
2	Technology Stack	2
3	Project Structure	3
3.1	Routing Philosophy	3
4	Application Architecture	3
4.1	Root Layout and Providers	4
4.1.1	Navigation Guard	4
4.2	Authentication (<code>context/AuthContext.tsx</code>)	4
4.2.1	User Model	4
4.2.2	Session Persistence and Restoration	5
4.2.3	Built-in Admin	5
4.3	Theming (<code>context/ThemeContext.tsx</code>)	5
4.4	Design System (<code>constants/theme.ts</code>)	5
5	Networking Layer	6
5.1	HTTP Client (<code>api/client.ts</code>)	6
5.1.1	Global 401 Handling	6
5.2	Endpoint Bindings (<code>api/devices.ts</code>)	6
5.2.1	Defensive Normalisation	7
5.2.2	Core Data Models	7
5.3	Runtime Server Configuration (<code>constants/apiConfig.ts</code>)	7
5.4	Client-Side Caching (<code>hooks/useCachedResource.ts</code>)	8
6	Screens	8
6.1	HomeScreen — Application Shell	8
6.1.1	Tab Model and Transitions	8
6.1.2	Device State and the Cabin Model	8
6.1.3	Optimistic Updates and Reconciliation	9
6.1.4	Additional Behaviours	9
6.2	LoginScreen & SignupScreen	9
6.3	SchedulesScreen — Automations	9
6.4	AnalyticsScreen — Uptime Timeline	10
6.5	LogsScreen — Snapshot History	10
6.6	SettingsScreen (Admin)	10
6.7	AdminUsersScreen — Cabin Assignment	10
7	Reusable Components	11
7.1	Notable Component Patterns	11
8	Cross-Cutting Concerns	12
8.1	State Management	12
8.2	Performance Techniques	12
8.3	Offline and Failure Resilience	12
8.4	Accessibility & UX Details	12
9	Build & Configuration	12
9.1	Running the App	13

10 Conclusion	13
----------------------	-----------

1 Introduction

Nestboard is the mobile front-end of the ESP Room Automation system. It provides a polished, real-time interface for monitoring and controlling electrical devices (lights and fans) distributed across a set of rooms (referred to in the UI as *cabins*). The application talks to a FastAPI backend over a REST API; the backend, in turn, relays device state changes to ESP32 microcontrollers through MQTT.

The client is a universal application: a single TypeScript/React Native codebase targets Android, iOS and the web through the Expo toolchain. This document describes the architecture, structure, screens, components and supporting infrastructure of the front-end.

The mobile application front-end and this accompanying document were designed and developed by **Abdullah Amir (2024-CS-156)**.

1.1 Feature Summary

- **Authenticated access** with token-based sessions, role separation (*admin* vs. *user*) and a built-in default admin account.
- **Live device control** — per-cabin light/fan toggles with optimistic UI updates and background reconciliation via polling.
- **Master controls** for turning all lights on, all fans on, or all devices off (with a hold-to-confirm safety gesture).
- **Scheduling** — time- and day-based automations targeting one or many devices.
- **Analytics** — a Gantt-style device-uptime timeline and per-room uptime statistics.
- **Activity logs** — historical state snapshots with an in-house line chart (no external charting dependency).
- **User management** — admins can create, delete, reset passwords for and assign cabins to users.
- **Theming** — light, dark and system-driven colour schemes.
- **Runtime server configuration** — the backend URL is editable from the login screen and persisted on device.

2 Technology Stack

Concern	Technology / Library
Framework	React 19 + React Native 0.81
Tooling / runtime	Expo SDK 54 (<code>expo</code> , <code>expo-router</code>)
Language	TypeScript 5.9 (strict mode)
Navigation	Expo Router (file-based) + React Navigation
Animations	<code>react-native-reanimated</code> 4, Animated API
Gestures	<code>react-native-gesture-handler</code>
Local persistence	<code>@react-native-async-storage/async-storage</code>
Icons	<code>@expo/vector-icons</code> (Ionicons)
Haptics	<code>expo-haptics</code>
Visual effects	<code>expo-linear-gradient</code> , <code>expo-blur</code>
Linting	ESLint 9 + <code>eslint-config-expo</code>

Concern	Technology / Library
Build / release	EAS (eas.json)

Package identity. The npm package is named `nestboard`; the Expo app is `Nestboard` with Android package `com.abdullahamir.Nestboard`. The New Architecture (`newArchEnabled`) and the React Compiler (`experiments.reactCompiler`) are both enabled.

3 Project Structure

The front-end lives under `app/SmartOffice`. It follows a conventional Expo Router layout: the `app/` directory defines routes, while the substantive UI lives in `screens/` and `components/`.

```

app/SmartOffice/
  app/
    # Expo Router routes (thin re-exports)
    _layout.tsx      # Root layout: providers + navigation guard
    login.tsx, signup.tsx # Auth routes
    (tabs)/
      _layout.tsx    # Tab group (native tab bar hidden)
      index.tsx      # -> HomeScreen
      analytics.tsx  # -> AnalyticsScreen
  screens/
    # Full-page UI
    HomeScreen.tsx   # Shell hosting all tabs + device grid
    LoginScreen.tsx, SignupScreen.tsx
    SchedulesScreen.tsx # Automations CRUD
    AnalyticsScreen.tsx # Uptime Gantt + stats
    LogsScreen.tsx     # State-snapshot history
    SettingsScreen.tsx # Appearance + user admin
    AdminUsersScreen.tsx # Cabin assignment
  components/
    # Reusable UI widgets
  context/
    # AuthContext, ThemeContext
  api/
    # client.ts (fetch wrapper), devices.ts (endpoints)
  constants/
    # theme, apiConfig, cabinData, acknowledgements
  hooks/
    # useCachedResource (SWR-style cache)
  assets/
    # icons, splash, profile images
  app.json, eas.json # Expo / EAS configuration

```

3.1 Routing Philosophy

Routes under `app/` are intentionally thin. For example, `app/(tabs)/index.tsx` is a single re-export:

```

import HomeScreen from '../..//screens/HomeScreen';
export default HomeScreen;

```

This keeps route files declarative and concentrates real logic in `screens/`. Notably, the native tab bar is hidden (`tabBarStyle: { display: 'none' }`); `HomeScreen` instead renders a custom `BottomNav` and manages inter-tab transitions itself (see Section 6.1).

4 Application Architecture

4.1 Root Layout and Providers

`app/_layout.tsx` composes the provider tree and installs a navigation guard. The hierarchy is:

```
<GestureHandlerRootView>
  <AuthProvider>
    <ThemeContextProvider>
      <ThemedNavigation /> // NavigationGuard + <Stack> + StatusBar
    </ThemeContextProvider>
  </AuthProvider>
</GestureHandlerRootView>
```

On mount, the root also fires a fire-and-forget `pingHealth()` against the backend to warm the connection.

4.1.1 Navigation Guard

`NavigationGuard` enforces authentication by comparing the current route segment against the user's login state:

```
const inAuthGroup = segments[0] === '(tabs)';
if (!user && inAuthGroup) router.replace('/login');
else if (user && !inAuthGroup) router.replace('/');
```

Unauthenticated users are redirected to `/login`; authenticated users are kept inside the `(tabs)` group. Redirection is deferred until the auth state finishes loading (`loading` flag) to avoid a flash of the wrong screen at startup.

4.2 Authentication (`context/AuthContext.tsx`)

`AuthContext` is the single source of truth for the signed-in user. It exposes:

Member	Purpose
<code>user</code>	Current <code>AppUser</code> or <code>null</code> .
<code>loading</code>	<code>true</code> while the persisted session is being restored.
<code>login</code>	Authenticates via <code>POST /login</code> (OAuth2 form).
<code>signup</code>	Registers via <code>POST /signup</code> .
<code>logout</code>	Clears the token and session from storage.

4.2.1 User Model

```
interface AppUser {
  id: string;
  name: string;
  email: string;
  password: string; // never populated after login
  role: 'admin' | 'user';
  assignedCabinIds: string[]; // e.g. ['cabin-1', 'cabin-3']
}
```

Backend "rooms" (`r1`, `r2`, ...) are mapped to UI "cabins" (`cabin-1`, `cabin-2`, ...) by `roomsToCabinIds()`.

4.2.2 Session Persistence and Restoration

Two keys are stored in `AsyncStorage`: a session username (`nestboard_session`) and a bearer token (`nestboard_token`). On startup the provider:

1. Reads both keys in parallel.
2. If the stored id is the built-in `ADMIN` , restores it directly.
3. Otherwise, fetches the live profile via `GET /me` so that role and room assignments are always current; a failure clears the session (expired token) and forces re-login.

4.2.3 Built-in Admin

A hard-coded administrator account is provided for first-run/bootstrap use:

```
const ADMIN = {
  id: 'admin', name: 'Admin', email: 'admin@nestboard.com',
  password: 'admin123', role: 'admin', assignedCabinIds: [],
};
```

Security note. The default admin credentials are embedded in the client bundle. For a production deployment these should be removed or replaced with a server-verified account, since anything shipped in the app binary is recoverable by an attacker.

4.3 Theming (context/ThemeContext.tsx)

`ThemeContextProvider` manages a three-way preference — `'light'` , `'dark'` or `'system'` — persisted under `nestboard_theme_pref` . When the preference is `'system'` it follows the device colour scheme via React Native's `useColorScheme()` .

The resolved `colors` object (from `constants/theme.ts`) is memoised and distributed through context, so every component re-styles automatically when the theme changes. The exposed API is `{ colors, theme, preference, setPreference, toggleTheme }` .

4.4 Design System (constants/theme.ts)

A small, explicit design system underpins the "glass"/frosted aesthetic used throughout the app.

Colour tokens. Two palettes — `darkColors` and `lightColors` — share an identical key set (enforced by `typeof darkColors`) so that any component works in both themes. Key roles:

Token	Role
<code>background</code>	/ Page and card backgrounds.
<code>surface</code>	
<code>glass</code> , <code>glassBorder</code> , <code>glassHighlight</code>	Translucent "frosted" fills, borders and top highlights.
<code>accent</code> , <code>accentLight</code> , <code>accentGlow</code>	Primary blue (#2F80ED) and its variants for active/glow states.
<code>text</code> , <code>textSecondary</code> , <code>textMuted</code>	Typographic hierarchy.

Token	Role
<code>success</code> , <code>toggleOff</code>	<code>blue</code> , Status and control colours.

Spacing and radii. Fixed scales keep layout rhythm consistent:

```
SPACING = { xs:4, sm:8, md:12, lg:16, xl:20, xxl:24, xxxl:32 };
RADIUS  = { sm:10, md:16, lg:20, xl:26, full:999 };
```

Every screen and component builds its `StyleSheet` through a `createStyles(colors)` factory wrapped in `useMemo`, which is the codebase's standard idiom for theme-aware styling.

5 Networking Layer

5.1 HTTP Client (`api/client.ts`)

A thin `fetch` wrapper exposes `get`, `post`, `put`, `del` and a specialised `postForm` (for OAuth2 `x-www-form-urlencoded` login/signup). Each request:

- resolves the base URL asynchronously (see §5.3);
- attaches `Authorization: Bearer <token>` when a token is present;
- routes the response through a shared `handle()` function.

5.1.1 Global 401 Handling

If an authenticated request returns `401`, the client invokes a globally registered `onUnauthorized` handler. `AuthContext` registers this handler to clear the session and drop the user back to the login screen — a clean, centralised expired-token flow:

```
async function handle(res, hadToken) {
  if (res.status === 401 && hadToken && onUnauthorized) onUnauthorized();
  if (!res.ok) throw res;      // raw Response is thrown on error
  return res.json();
}
```

Because the raw `Response` is thrown on non-2xx, callers can inspect `status` and read `detail` from the JSON body (a pattern used throughout for error messages).

5.2 Endpoint Bindings (`api/devices.ts`)

This module is the typed contract with the backend. It defines the data interfaces and one function per endpoint. Highlights:

Function	Endpoint	Purpose
<code>pingHealth</code>	<code>GET /health</code>	Connectivity warm-up.
<code>getStates</code>	<code>GET /states</code>	Device states + room names.
<code>setDevice</code>	<code>POST /set/{id}</code>	Set one device on/off.
<code>setAll</code>	<code>POST /set-all</code>	Set every visible device.
<code>lightsOn/fansOn/allOff</code>	<code>POST /lights/on ...</code>	Master actions.

Function	Endpoint	Purpose
<code>listRooms/renameRoom</code>	GET/PUT <code>/rooms</code>	Room name management.
<code>getLogs</code>	GET <code>/logs</code>	Recent state snapshots.
<code>getLogsRange</code>	GET <code>/logs/range</code>	Bucketed chart data.
<code>triggerLog</code>	POST <code>/logs/trigger</code>	Capture a snapshot now.
<code>listSchedules ...</code>	<code>/schedules</code>	Schedule CRUD.
<code>listUsers ...</code>	<code>/users</code>	User admin.
<code>assignUserRooms</code>	PUT <code>/users/{u}/rooms</code>	Cabin assignment.
<code>changePassword</code>	POST <code>/change-password</code>	Password reset.

5.2.1 Defensive Normalisation

The layer is deliberately tolerant of backend shape drift. Two examples:

- `getStates` accepts both the new `{ states, names }` shape and the legacy flat map, always returning the new shape.
- `normalizeSchedule` guarantees `device_ids` is always a `string[]`, tolerating a legacy singular `device_id` field.

5.2.2 Core Data Models

```
interface StatesResponse {
  states: Record<string, number>; // 'r1/l1' -> 0/1
  names: Record<string, string>; // 'r1' -> 'Reception'
}

interface Schedule {
  id: number; device_ids: string[]; action: number; // 1=on, 0=off
  hour: number; minute: number; days: string[]; // ['mon',...]
  enabled: boolean; created_by: string; created_at: string;
}

interface StateLogEntry {
  id: number; logged_at: string; snapshot: Record<string, number>;
}
```

5.3 Runtime Server Configuration (`constants/apiConfig.ts`)

The backend base URL is not hard-coded at build time. Instead it is:

- defaulted to `http://192.168.0.127:8000`;
- overridable at runtime from the login screen and persisted under `nestboard_base_url`;
- hydrated once into an in-memory cache so request code can read it synchronously after startup.

`normalizeBaseUrl()` is lenient: it accepts a bare IP, an `IP:port`, or a full URL, and infers the scheme — plain `http` for LAN addresses/ `localhost`, `https` otherwise — before stripping any trailing slash. This makes it easy to point the app at different deployments (local hardware, a tunnel, or a hosted server) without rebuilding.

5.4 Client-Side Caching (`hooks/useCachedResource.ts`)

A lightweight stale-while-revalidate hook backs the read-heavy screens (logs, charts). It layers three tiers:

1. **In-memory** map for instant reads across tab switches.
2. **AsyncStorage** for persistence across app restarts.
3. **TTL** so the network is only re-hit once a copy is stale.

The hook returns `{ data, loading, refreshing, refresh, reload, setData }`. It seeds synchronously from the memory cache (so return visits render instantly), supports pull-to-refresh (`refresh`, always networks) and a silent `reload` after mutations, and can be gated with an `active` flag so an off-screen tab performs no work.

6 Screens

6.1 HomeScreen — Application Shell

`HomeScreen` is the most substantial screen; it is both the device dashboard and the host container for every other tab. Rather than mounting separate routes, it holds all tab screens as horizontally-stacked `Animated.View` pages and slides between them.

6.1.1 Tab Model and Transitions

Visible tabs depend on role. Admins see six (`home`, `schedules`, `analytics`, `users`, `settings`, `logs`); regular users see two (`home`, `schedules`). A single Reanimated shared value `offset` drives every page's `translateX`, producing a smooth carousel:

```
offset.value = withTiming(tabOffset(activeTab, isAdmin), TIMING);
// each page: translateX = (position - offset) * SCREEN_WIDTH
```

Navigation is possible both via the `BottomNav` and via horizontal swipe, implemented with a `PanResponder` that claims the gesture only when horizontal motion clearly dominates. Off-screen tabs are lazily mounted the first time they are visited (`visitedTabs` set), avoiding four screens' worth of fetches at launch.

6.1.2 Device State and the Cabin Model

The grid is seeded from `INITIAL_CABINS` (six cabins, each with a light and a fan) defined in `constants/cabinData.ts`:

```
interface Cabin {
  id: string; name: string; number: number;
  light: CabinDevice; // { id, label, isOn, topic: 'r1/l1' }
  fan: CabinDevice; // { id, label, isOn, topic: 'r1/f1' }
}
```

Each device carries an MQTT-style `topic` (`r{n}/l1`, `r{n}/f1`) that maps it to a backend device id. Non-admin users see only their `assignedCabinIds`.

6.1.3 Optimistic Updates and Reconciliation

Toggling a device updates local state immediately, then issues `setDevice()`. To prevent an in-flight poll from "resurrecting" the old value, the screen maintains a `pendingRef` map of topics to their expected settled value:

- On toggle, the topic is marked pending with the expected 0/1, with an 8-second safety timeout.
- A background poll runs every 5 seconds calling `getStates()` and `reconcileStates()`.
- `reconcileStates` ignores incoming values for a pending topic until they match the expectation, then releases the lock — eliminating flicker.
- Server-provided room names override the friendly defaults.

To minimise re-renders, `applyCabinUpdate` reuses the previous cabin object (and the previous array) whenever nothing actually changed, so idle polls cost zero renders and `CabinCard` (memoised) only re-renders for the cabin that changed.

6.1.4 Additional Behaviours

- **Drag-to-reorder** grid, with the order persisted under `nestboard_cabin_order` and restored on mount.
- **Master control** (admin only) for bulk actions.
- **Rename** of a cabin (optimistic, reverts on failure) via `renameRoom`.
- An **"Awaiting Assignment"** empty state for users with no cabins.

6.2 LoginScreen & SignupScreen

Both are form screens sharing the frosted-card aesthetic, keyboard-avoiding layout and animated entrance (`FadeInView`). `LoginScreen` additionally hosts the collapsible **Server settings** panel that reads and writes the runtime base URL (§5.3). Validation is performed inline; network/credential errors are surfaced with specific messages (e.g. invalid credentials vs. unreachable server, the latter including the target URL). `SignupScreen` enforces matching passwords and a 4-character minimum, and explains that an admin will assign a cabin after registration. Navigation after success is handled implicitly by the root `NavigationGuard`.

6.3 SchedulesScreen — Automations

The largest feature screen. It lists automations as cards and provides a bottom-sheet modal form to create or edit them.

- **Device source of truth.** The device picker is populated from `GET /states` keys, so it never offers a non-existent device (which would fail creation). `ALL_DEVICES` (rooms 1–6) is only a fallback.
- **Multi-device targeting.** A new schedule can target one, several or all available devices via an inline multi-select with "Select all". On edit, the device set is read-only (the backend forbids reassignment).
- **Time selection** uses the custom `WheelTimePicker` (hour / minute / AM/PM).
- **Day selection** via M–S chips; **action** is a Turn On / Turn Off segmented control; an **enabled** toggle completes the form.

- **Role scoping.** Non-admins only see and can only target schedules touching their assigned devices.
- **Robust errors.** `describeError` extracts FastAPI `detail` strings and validation arrays into readable alerts.
- Enabling/disabling a schedule is optimistic and reverts on failure.

6.4 AnalyticsScreen — Uptime Timeline

Consumes `GET /logs` (up to 500 snapshots) and renders, without any charting library:

- A **totals row** (active now, total devices, snapshot count).
- A **Gantt chart**: for each selected device, "on" intervals are reconstructed from consecutive snapshots and drawn as bars against a time axis. Selectable ranges are 6h, 12h, 1d, 7d and 30d, with adaptive axis labels and room separators.
- **Per-room stat tiles** showing uptime percentage and "last seen", colour-coded by uptime, tappable to add/remove a device from the chart.

Data refreshes every 30 seconds and on pull-to-refresh; a `triggerLog()` is fired on mount so the newest state is captured. All interval and statistics maths (`buildIntervals` , `computeStats` , `groupByRoom`) is pure and local.

6.5 LogsScreen — Snapshot History

A `FlatList` of state snapshots, newest first, each an expandable card showing the on/off chips for every device and an on-count badge; the latest entry is highlighted. The list header embeds `LogsChart` . A camera button triggers an on-demand snapshot (`triggerLog`) and force-reloads the cache. The screen is backed by `useCachedResource` (30s TTL) and only fetches when its tab is active. The list is tuned for performance (`initialNumToRender` , `windowSize` , `removeClippedSubviews` , memoised rows).

6.6 SettingsScreen (Admin)

Composed of three cards:

1. **Appearance** — light / dark / system theme selector wired to `ThemeContext` .
2. **Create User** — username + password form calling `createUser` , with inline validation and success/error feedback. Its state is reset when the tab loses focus (via a `formKey`).
3. **Manage Users** — list with per-row password-reset (opening `ChangePasswordModal`) and delete (with confirmation) actions.

Changes propagate to the sibling Users tab through an `onUserChanged` callback that bumps a refresh key on `HomeScreen` .

6.7 AdminUsersScreen — Cabin Assignment

Lists every registered user with avatar (initials), role and assigned-cabin chips. Tapping **Assign** opens a bottom-sheet modal that multi-selects cabins; on save, cabin ids are converted back to room ids (`cabin-1` → `r1`) and persisted via `assignUserRooms` , with the local list updated optimistically. It re-fetches when its `refreshKey` prop changes (driven by Settings actions).

7 Reusable Components

Component	Responsibility
<code>GlassCard</code>	Frosted container with border, translucent fill and a 1px top highlight; <code>highlighted</code> accents active cards.
<code>CabinCard</code>	Per-cabin tile: number badge, name, active/idle status, light & fan rows with toggles. Memoised; icons "pulse" on state change; press provides a scale animation.
<code>DraggableCabinGrid</code>	2-column sortable grid using Gesture Handler + Reanimated. Long-press lifts a card (with haptic), reorders via absolute positioning, and reports the new order. Freezes page scroll while dragging.
<code>ExpandedCabinModal</code>	Enlarged single-cabin view with larger toggles and inline rename (when permitted).
<code>MasterControl</code>	Three bulk buttons. "All Off" is a hold-to-confirm (2s) action with an animated progress fill and strong haptic on confirm, guarding against accidental taps.
<code>ToggleSwitch</code>	Animated on/off switch (<code>sm</code> / <code>md</code>) with spring thumb travel, colour cross-fade and a glow dot.
<code>BottomNav</code>	Custom pill navigation bar; role-filtered tabs, animated active indicator, filled icon + label on selection.
<code>Header</code>	Home header: greeting, theme toggle, logout, an info button (opens Acknowledgements) and a device/cabin stats row.
<code>WheelTimePicker</code>	iOS-alarm-style scroll wheels (hour/minute/AM-PM) with fade + scale falloff, snapping and selection haptics.
<code>LogsChart</code>	Dependency-free line chart (average + dashed peak series) drawn with rotated <code>View</code> segments, an area fill, gridlines, tap tooltips and a legend.
<code>ChangePasswordModal</code>	Admin password-reset sheet calling <code>changePassword</code> .
<code>AcknowledgementsModal</code>	Full-screen credits page (project info, contributors with avatars/LinkedIn, supervisor).
<code>FadeInView</code>	Mount-triggered fade + slide-up wrapper used app-wide for staggered entrances.
<code>Loader</code>	Branded spinner with pulsing glow and rotating status lines.

7.1 Notable Component Patterns

Hold-to-confirm (`MasterControl`). The destructive "All Off" button must be pressed for 2 seconds. A short vibration acknowledges the press, an animated fill tracks hold progress, and a strong buzz plus warning haptic fire on confirm. A quick tap only shows a "Hold 2s" hint — accidental taps cannot trigger it.

Draggable grid worklets. `DraggableCabinGrid` performs its layout math (`getPosition` , `getOrder`) inside Reanimated worklets on the UI thread, keeping dragging at 60 fps. A shared `positions` map is mutated during the gesture; the JS thread is only notified on drop.

No-SVG charting. Both `LogsChart` and the Analytics Gantt draw entirely with plain `View`s — line segments are thin views rotated by `atan2` angle — avoiding `react-native-svg` and its native build cost.

8 Cross-Cutting Concerns

8.1 State Management

The app uses **React Context** for global concerns (auth, theme) and **local component state** for everything else — there is no Redux/MobX. Data freshness is achieved through a combination of polling (Home, Analytics) and the stale-while-revalidate cache (Logs, chart).

8.2 Performance Techniques

- Memoised style factories (`useMemo(() => createStyles(colors))`).
- `React.memo` on list rows (`CabinCard` , `log cards`).
- Structural sharing in `applyCabinUpdate` to skip no-op renders.
- Lazy tab mounting and `active`-gated data fetching.
- Tuned `FlatList` windowing.
- UI-thread animations via Reanimated worklets and native-driver `Animated`.

8.3 Offline and Failure Resilience

Optimistic updates keep the UI responsive under latency; failed mutations either revert (rename, schedule enable) or are reconciled by the next poll (toggles). Cached resources continue to display the last-known data when the network is unreachable. A `401` anywhere triggers a clean, global logout.

8.4 Accessibility & UX Details

Consistent hit areas, haptic feedback on meaningful interactions, theme-aware status bars, keyboard-avoiding forms, pull-to-refresh on all lists, and staggered entrance animations contribute to a native-feeling experience.

9 Build & Configuration

File	Role
<code>app.json</code>	Expo config: name, slug, icons, splash, Android package, plugins (<code>expo-router</code> , splash, build-properties), New Architecture and React Compiler flags.
<code>eas.json</code>	EAS Build/Submit profiles.
<code>tsconfig.json</code>	Extends <code>expo/tsconfig.base</code> , strict mode, <code>@/*</code> path alias.
<code>package.json</code>	Scripts: <code>start</code> , <code>android</code> , <code>ios</code> , <code>web</code> , <code>lint</code> .

Cleartext traffic. `expo-build-properties` enables `usesCleartextTraffic` on Android so the app can reach a plain- `http` LAN backend (e.g. ESP hardware on the local network). This is appropriate for the project's LAN deployment but should be reconsidered for public release.

9.1 Running the App

```
cd app/SmartOffice
npm install
npx expo start      # then press a (Android), i (iOS), or w (web)
```

At first launch, open **Server settings** on the login screen and point the app at the backend (default `http://192.168.0.127:8000`), then sign in with the admin account or a provisioned user.

10 Conclusion

The Nestboard front-end is a cohesive, single-codebase React Native application that delivers real-time room automation with a refined, theme-aware interface. Its architecture cleanly separates routing, global state, a typed networking layer and a reusable component library. Careful attention to optimistic updates, reconciliation, caching and render minimisation gives it a responsive feel even over an intermittent LAN link to embedded hardware. The result is a maintainable, extensible client suitable both as a course deliverable and as a foundation for further development.

— End of Document —